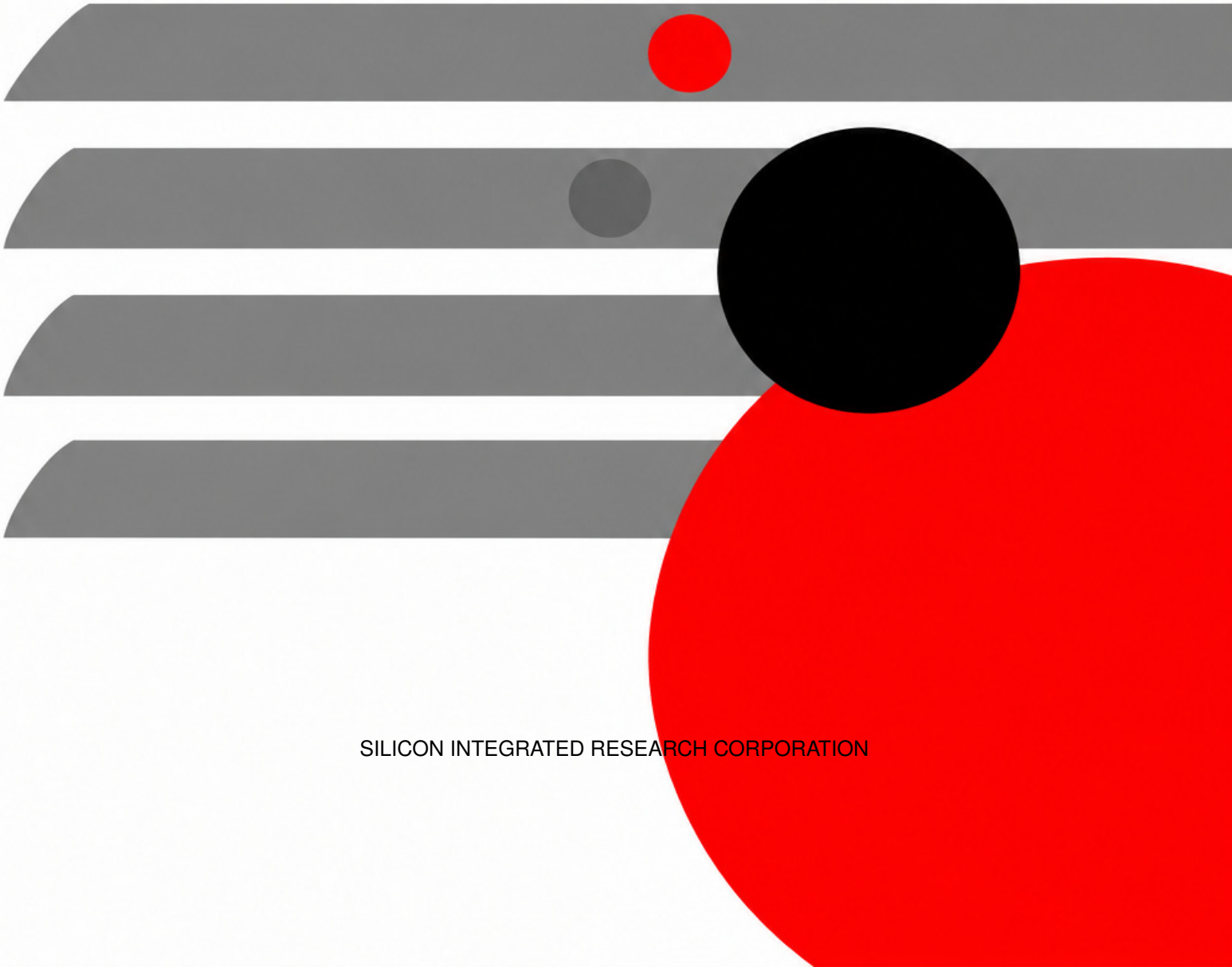



# SIRC-1

## Hardware Reference Manual

*Package, Signals and Bus Interface*



SILICON INTEGRATED RESEARCH CORPORATION

# Preface

## About This Manual

This manual documents the SIRC-1 CPU as a hardware component: its package, its signals, and the bus protocol it uses to talk to memory and peripherals. It is the companion to the *SIRC-1 Architecture Reference Manual*, which defines the instruction set, registers, exceptions and other programmer-visible behavior. Everything in this manual is about what happens on the pins; the *SIRC-1 Architecture Reference Manual* is about what the processor does.

It is written for:

- Board and system designers
- Memory controller and peripheral designers
- Hardware implementers building a compatible device

## Assumed Knowledge

This manual assumes familiarity with digital logic, synchronous bus protocols, and active-low signaling. Readers who also need to know how the processor responds to resets, interrupts and bus faults should read the corresponding chapters of the *SIRC-1 Architecture Reference Manual*.

## Revision Status

This is Version 1.0 of the *SIRC-1 Hardware Reference Manual*, issued in draft for internal review and not for external distribution. Chapter 6 lists the electrical characteristics that have not yet been specified.

# Contents

**Preface** iii

**1 Introduction** 1
1.1 Purpose and Scope . . . 1
1.2 Relationship to the Architecture Manual . . . 1
1.3 Signal Groups . . . 1

**2 Package and Pinout** 3
2.1 Chip Layout . . . 3

**3 Signal Descriptions** 5
3.1 Signal Conventions . . . 5
3.2 Address Bus Bits (A0–A23) . . . 5
3.3 Power Pins (VCC1–VCC2, GND1–GND2) . . . 5
3.4 Data Bus Bits (D0–D15) . . . 5
3.5 Clock Input (CLKI) . . . 5
3.6 Instruction Sync Output (SYNC) . . . 6
3.7 Bus Access Strobe Output (BAS) . . . 6
3.8 Bus Read/Write Output (BRW) . . . 6
3.9 Bus Acknowledge Input (BACK) . . . 6
3.10 Bus Error Input (BERR) . . . 6
3.11 Bus Protection Error Input (BPER) . . . 7
3.12 Bus Access Type Bits (BAT0–BAT2) . . . 7
3.13 Protected-Mode Output (PROT) . . . 7
3.14 External CPU Halt Input (HALT) . . . 8
3.15 Force Trace Mode Input (TRCE) . . . 8
3.16 Maskable Interrupt Inputs (IRQ1–IRQ4) . . . 8
3.17 Non-Maskable Interrupt Input (NMI) . . . 9
3.18 Reset Input (RSTI) . . . 9
3.19 Reset Output (RSTO) . . . 9

**4 Bus Interface** 11
4.1 Clocked Bus Model . . . 11
4.2 Bus Cycle Completion . . . 12
4.3 Bus Cycle Types . . . 12
4.4 DMA Bus Cycles . . . 12
4.5 Control Input Sampling . . . 14

**5 Bus Timing Diagrams** 15
5.1 Bus Cycle Diagrams . . . 15
5.2 Reset and Interrupt Timing Examples . . . 18

**6 Electrical Characteristics** 21

# List of Tables

1.1 The SIRC Manual Set . . . . . 1
1.2 SIRC-1 Signal Groups . . . . . 1

2.1 SIRC CPU 64-pin DIP Pinout . . . . . 4

3.1 Bus Access Type Bits Meaning . . . . . 7

4.1 CPU Bus Output Timing Rules . . . . . 11
4.2 Bus Response Timing Rules . . . . . 12
4.3 External Bus Cycle Ownership . . . . . 12
4.4 External Input Sampling Rules . . . . . 14

# List of Figures


2.1 SIRC CPU 64-pin DIP Pinout, Top View . . . . . 3

4.1 Three-Word DMA Read with No Wait States . . . . . 13

5.1 Instruction-Fetch High-Word and Low-Word Reads with No Wait States . . . . . 15
5.2 Data Read Cycle with No Wait State . . . . . 16
5.3 Data Write Cycle with No Wait State . . . . . 16
5.4 Exception-Vector Fetch with High-Word Then Low-Word Reads . . . . . 16
5.5 Wait-State Insertion when No Bus Response Is Asserted . . . . . 17
5.6 Bus-Error and Bus-Protection Abort Timing . . . . . 17
5.7 Complete Register-Only ALU Instruction with No Wait States . . . . . 17
5.8 Complete LOAD Instruction with No Wait States . . . . . 18
5.9 Hardware Reset with One-Cycle $\overline{\text{RSTI}}$ Assertion and Reset-Vector Fetch . . . . . 18
5.10 Enabled $\overline{\text{IRQ}}$ Latched Before an Instruction Boundary and Dispatched with No Wait States . . . . . 19

CHAPTER 1

# Introduction

## 1.1 Purpose and Scope

The SIRC-1 documentation is split into three manuals, each with its own scope:

| Manual | Scope |
|---|---|
| *SIRC-1 Architecture Reference Manual* | ISA, instruction encodings, canonical assembly notation, exceptions, programmer-visible state |
| *SIRC-1 Assembly Programmer's Manual* | Complete assembly syntax, directives, symbols, meta-instructions, and toolchain usage |
| *SIRC-1 Hardware Reference Manual* | Pinout, bus interface, timing, electrical characteristics (this manual) |

Table 1.1: The SIRC Manual Set

This manual covers the concrete silicon: the 64-pin package, what each signal does, the bus protocol, and the timing of bus cycles, resets and interrupts. It does not describe instructions, registers or exception handling from the programmer's point of view.

## 1.2 Relationship to the Architecture Manual

The *SIRC-1 Architecture Reference Manual* defines what the CPU does: which instructions exist, how exceptions are raised and returned from, and what the programmer can observe. This manual defines how that behavior appears at the pins. Several behaviors are visible in both places, for example the reset sequence and the bus fault retry rules. Where they overlap, the *SIRC-1 Architecture Reference Manual* is authoritative for what happens inside the CPU and this manual is authoritative for what is driven or sampled on each signal.

## 1.3 Signal Groups

The 64 pins fall into the groups below. Chapter 2 gives the pin numbers and Chapter 3 describes every signal.

| Group | Signals | Pins |
|---|---|---|
| Address bus | A0–A23 | 24 |
| Data bus | D0–D15 | 16 |
| Bus control | $\overline{\text{BAS}}$, BRW, BAT0–BAT2, $\overline{\text{BACK}}$, $\overline{\text{BERR}}$, $\overline{\text{BPER}}$, PROT, SYNC | 10 |
| Interrupts and execution control | $\overline{\text{IRQ1}}$–$\overline{\text{IRQ4}}$, $\overline{\text{NMI}}$, $\overline{\text{HALT}}$, $\overline{\text{TRCE}}$ | 7 |
| Reset | $\overline{\text{RSTI}}$, $\overline{\text{RSTO}}$ | 2 |
| Clock | CLKI | 1 |
| Power | VCC1–VCC2, GND1–GND2 | 4 |

Table 1.2: SIRC-1 Signal Groups

CHAPTER 2

# Package and Pinout

## 2.1 Chip Layout

The SIRC-1 is provided in a 64-pin dual in-line package (DIP-64).

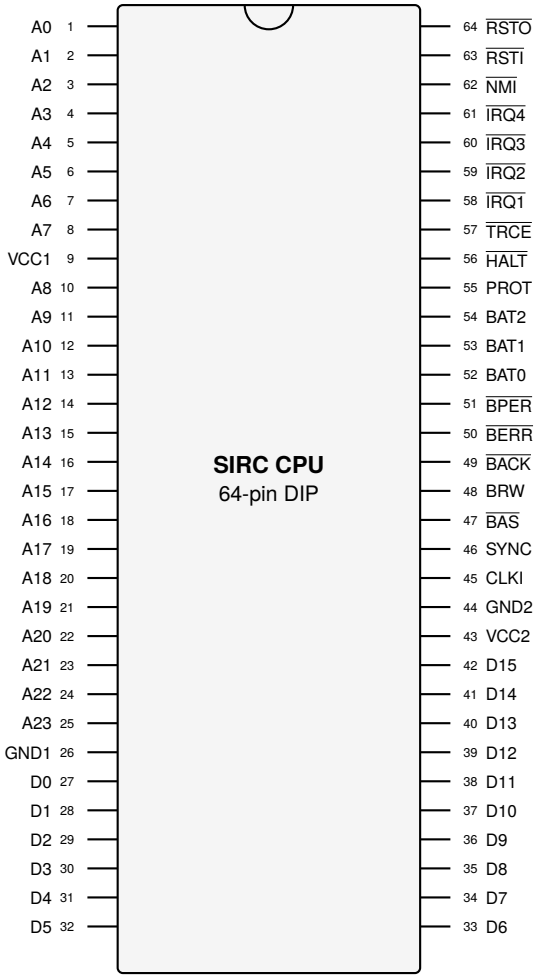



Figure 2.1: SIRC CPU 64-pin DIP Pinout, Top View

| Pin | Name | Description |
|---|---|---|
| 1–8 | A0–A7 | Address bus bits 0–7 |
| 9 | VCC1 | Power supply |
| 10–25 | A8–A23 | Address bus bits 8–23 |
| 26 | GND1 | Ground |
| 27–42 | D0–D15 | Data bus bits 0–15 |
| 43 | VCC2 | Power supply |
| 44 | GND2 | Ground |
| 45 | CLKI | Clock input |
| 46 | SYNC | Instruction-sync output |
| 47 | $\overline{\text{BAS}}$ | Bus access strobe output |
| 48 | BRW | Bus read/write output |
| 49 | $\overline{\text{BACK}}$ | Bus acknowledge input |
| 50 | $\overline{\text{BERR}}$ | Bus error input |
| 51 | $\overline{\text{BPER}}$ | Bus protection error input |
| 52–54 | BAT0–BAT2 | Bus access type bits 0–2 |
| 55 | PROT | Protected-mode output |
| 56 | $\overline{\text{HALT}}$ | External CPU halt input |
| 57 | $\overline{\text{TRCE}}$ | Force trace mode input |
| 58–61 | $\overline{\text{IRQ1}}$–$\overline{\text{IRQ4}}$ | Maskable interrupt inputs 1–4 |
| 62 | $\overline{\text{NMI}}$ | Non-maskable interrupt input |
| 63 | $\overline{\text{RSTI}}$ | Reset input |
| 64 | $\overline{\text{RSTO}}$ | Reset output |

Table 2.1: SIRC CPU 64-pin DIP Pinout

CHAPTER 3

# Signal Descriptions

## 3.1 Signal Conventions

Each signal lists its own assertion convention. Address, data, and bus-type value pins use positive logic, while bus control, handshake, interrupt, halt, trace, and reset pins generally use active-low conventions. The descriptions below use "asserted" and "deasserted" to refer to the logical state of a signal according to its signal type.

## 3.2 Address Bus Bits (A0–A23)

- **Direction:** Output
- **Signal Type:** Active High

Connected to the address bus.

These 24 pins declare the address on the bus that the CPU reads from or writes to when $\overline{\text{BAS}}$ (bus access strobe) is asserted.

## 3.3 Power Pins (VCC1–VCC2, GND1–GND2)

These pins provide the 5 V power that the CPU requires to function.

The VCC pins must be connected to 5 V $\pm$ 5%, and GND pins must be connected to a 0 V ground rail.

## 3.4 Data Bus Bits (D0–D15)

- **Direction:** Bidirectional
- **Signal Type:** Active High

Connected to the data bus.

These 16 pins are driven to a 16-bit value when the CPU is outputting to the bus, and are an input for a 16-bit value when the CPU is reading from the bus.

## 3.5 Clock Input (CLKI)

- **Direction:** Input
- **Signal Type:** Rising Edge

Progresses the CPU to the next cycle when the input transitions from low to high (rising edge).

Each rising edge of CLKI advances the CPU by one execution phase, except while a bus wait state holds the phase, while the CPU is halted or waiting (**`WAIT`**), or while it is held in reset. With no wait states, an ordinary instruction takes six cycles. Coprocessor calls take longer; see the Instruction Timing appendix of the *SIRC-1 Architecture Reference Manual*.

## 3.6 Instruction Sync Output (SYNC)

- **Direction:** Output
- **Signal Type:** Active High

Asserted only at phase 0 of an instruction fetch or of an exception-unit dispatch slot. It remains asserted if a wait state holds that phase. It is deasserted in every other phase, in every other coprocessor slot (DMA and maths), in every DMA continuation slot, and while the CPU is halted, waiting, or in reset.

This pin is useful for debugging.

## 3.7 Bus Access Strobe Output (BAS)

- **Direction:** Output
- **Signal Type:** Active Low

When the CPU is ready to perform a bus operation, $\overline{\text{BAS}}$ is asserted after the address pins, BRW, BAT, PROT, and any CPU-driven data pins are valid.

$\overline{\text{BAS}}$ remains asserted until the bus responds with $\overline{\text{BACK}}$, $\overline{\text{BERR}}$, or $\overline{\text{BPER}}$. If no response is asserted, the CPU holds the current execution phase and repeats the same bus request on the following clock.

## 3.8 Bus Read/Write Output (BRW)

- **Direction:** Output
- **Signal Type:** Read High / Write Low

Only relevant when the $\overline{\text{BAS}}$ (Bus access strobe output) is asserted.

When high, the CPU is requesting a read from the bus. When low, the CPU is requesting a write to the bus.

## 3.9 Bus Acknowledge Input (BACK)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write completed successfully. On reads, the CPU samples D0–D15 on the clock edge that accepts $\overline{\text{BACK}}$.

If all bus devices are fast and can complete reads/writes in a single cycle, this pin can be permanently held asserted.

## 3.10 Bus Error Input (BERR)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was unsuccessful.

This causes the CPU to abort the I/O operation and raise a bus fault exception. A failed fault-vector fetch raises a double fault instead, a failed double-fault vector fetch resets the CPU, and a failed reset-vector fetch re-enters reset (see the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*).

For example, a bus error might be raised by I/O to an address that is not connected to any device.

## 3.11 Bus Protection Error Input (BPER)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was to a valid address/device, but that the I/O is disallowed given the current state of the CPU or bus.

This causes the CPU to abort the I/O operation and raise a bus protection fault exception. A failed fault-vector fetch raises a double fault instead, a failed double-fault vector fetch resets the CPU, and a failed reset-vector fetch re-enters reset (see the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*).

It behaves much like the $\overline{\text{BERR}}$ pin but raises a different fault so that programs can handle it differently.

For example, a bus protection error might be raised by a write to ROM, or by a write to a privileged section of memory when the CPU is not in supervisor mode.

## 3.12 Bus Access Type Bits (BAT0–BAT2)

- **Direction:** Output
- **Signal Type:** Active High

Output by the CPU to describe what I/O operation it is performing.

These pins can provide hints to external devices during I/O.

For example, if a memory controller knows that the DMA coprocessor will be reading more than one sequential address, it can prefetch the next address.

Another example is a memory controller providing memory protection when these pins are used with the PROT (Protected-mode output) pin. A memory controller might assert $\overline{\text{BPER}}$ (bus protection error) if BAT is 010 (data read), PROT is 1 (protected mode) and the address is in a protected memory region.

| BAT2 | BAT1 | BAT0 | Description |
|---|---|---|---|
| 0 | 0 | 0 | No I/O |
| 0 | 0 | 1 | Instruction Fetch |
| 0 | 1 | 0 | Data Read |
| 0 | 1 | 1 | Data Write |
| 1 | 0 | 0 | Exception Vector Fetch |
| 1 | 0 | 1 | DMA Coprocessor Read Burst |
| 1 | 1 | 0 | DMA Coprocessor Write Burst |
| 1 | 1 | 1 | Reserved |

Table 3.1: Bus Access Type Bits Meaning

## 3.13 Protected-Mode Output (PROT)

- **Direction:** Output
- **Signal Type:** Active High

Indicates whether the CPU is in protected mode (high) or in supervisor mode (low).

Follows the status register Protected Mode (P) bit, except that it is deasserted for every exception-vector fetch (BAT 100), which is a supervisor access even when the exception is taken from protected mode.

When the CPU is in protected mode, programs have access to fewer types of instructions, for example they cannot write to the high word of the address registers.

Exposing this as an external pin allows other devices such as memory controllers to block access to protected regions of memory by raising bus protection errors.

## 3.14 External CPU Halt Input (HALT)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU stops executing instructions.

$\overline{\text{HALT}}$ is sampled at every phase 0. If it is asserted there, the CPU stops at that phase 0 with the bus idle. The halt can occur between an instruction and its coprocessor dispatch slot, between DMA slots, or before a fault, interrupt, or reset dispatch. Halt latency is at most six cycles. The CPU performs no bus cycles while halted. The interrupt lines are still sampled and a Level 5 edge is still latched, but nothing is dispatched until the CPU resumes. The CPU resumes exactly where it stopped when the input is deasserted. A reset ends the halt. The six-cycle $\overline{\text{RSTO}}$ hold runs first, and if $\overline{\text{HALT}}$ is still asserted after it, the CPU halts at the first phase 0, before the reset-vector fetch.

This pin is useful for hardware-driven debugging, as the CPU can be halted and the state of the system examined.

## 3.15 Force Trace Mode Input (TRCE)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted at an instruction boundary, the instruction being started is traced regardless of whether software has cleared the Trace Mode (T) flag in the status register. No trace fault is raised while a fault handler runs (exception level 7), since it would escalate to a double fault.

In trace mode, the CPU raises an exception on every instruction to allow software debugging.

## 3.16 Maskable Interrupt Inputs (IRQ1–IRQ4)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the corresponding hardware interrupt line is sampled by the CPU.

The lines can be masked by the Hardware Interrupt Enable Bits (E1–E4) of the status register.

Interrupt inputs are level-sensitive. They are sampled on every clock cycle except during reset: from the cycle in which reset is requested until the reset-vector fetch completes, no interrupt line, including $\overline{\text{NMI}}$, is sampled. Enabled interrupt lines are latched in the pending hardware-exceptions register; disabled maskable lines are ignored and not queued, and a line is not latched while its own handler or a higher-priority one is running. The requesting device must hold the line asserted until its handler acknowledges it; an assertion withdrawn before it is latched is not recognized. If an interrupt pin remains asserted after its handler returns, it may be sampled again through the normal interrupt rules.

See the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual* for more details.

## 3.17 Non-Maskable Interrupt Input (NMI)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU samples the Level 5 hardware exception line. Unlike $\overline{\text{IRQ1}}$–$\overline{\text{IRQ4}}$, which are level-sensitive, $\overline{\text{NMI}}$ is edge-triggered: it fires once on assertion regardless of how long the pin is held, and the pin must be released and reasserted to fire again. An $\overline{\text{NMI}}$ held asserted across a reset does not fire afterward, because reset does not clear the edge detector's last sampled level and so no transition occurs; the pin must be released and reasserted.

This interrupt cannot be masked. A new assertion of this pin (a fresh edge) while a Level 5 handler is running raises a Level Five Hardware Exception Conflict fault; holding the pin asserted from the edge that started the handler does not. A further edge while a Level 5 request is still pending (for example, behind a fault handler) is lost and is reported by a single conflict fault when Level 5 is next selected.

Because $\overline{\text{NMI}}$ is edge-triggered, holding the pin asserted after its handler returns does not by itself cause it to fire again. The pin must be released (driven high) for at least one CLKI cycle and asserted (driven low) again; the request fires on the clock edge at which the assertion is first sampled. Level 5 exception handlers must therefore execute quickly, and $\overline{\text{NMI}}$ should be asserted only for important, non-interruptible events.

## 3.18 Reset Input (RSTI)

- **Direction:** Input
- **Signal Type:** Active Low

External devices can use this pin to force a reset of the CPU.

When asserted, the CPU immediately aborts the current instruction or bus wait, stops normal processing, resets to phase 0, and begins the reset sequence described in the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*.

The $\overline{\text{RSTO}}$ pin is also asserted.

The reset-output hold lasts for as long as $\overline{\text{RSTI}}$ is asserted plus five cycles. The cycle in which $\overline{\text{RSTI}}$ is first sampled asserted counts as the first, and $\overline{\text{RSTO}}$ stays asserted for five further cycles after $\overline{\text{RSTI}}$ is sampled deasserted, so an $\overline{\text{RSTI}}$ assertion of $n$ cycles gives $n + 5$ cycles of $\overline{\text{RSTO}}$ (six for a one-cycle assertion). A software reset and a reset-vector fetch retry each give a six-cycle hold.

When this pin is deasserted, the CPU completes the reset-output hold, stops asserting the $\overline{\text{RSTO}}$ pin, and the exception unit fetches the reset vector.

If a system needs external devices to be reset for longer than the CPU reset-output hold, reset timing must be managed externally to the CPU, and $\overline{\text{RSTI}}$ must remain asserted for as long as the other devices need to reset.

## 3.19 Reset Output (RSTO)

- **Direction:** Output
- **Signal Type:** Active Low

Asserted while the $\overline{\text{RSTI}}$ pin is asserted and during the reset-output hold before reset-vector fetch.

It is also asserted for the reset-output hold after a software reset, after a reset caused by a failed double-fault vector fetch, and for a further hold before the CPU retries a reset-vector fetch that failed. Each of these holds is six CLKI cycles. A reset caused by a failed double-fault vector fetch behaves as a software reset: the six-cycle $\overline{\text{RSTO}}$ hold is followed by the reset-vector fetch.

A reset can be triggered by asserting the Reset input ($\overline{\text{RSTI}}$) pin, or in software by the exception coprocessor meta-instruction **`RSET`**.

This pin is useful to reset other devices on the bus whenever the CPU is reset or to trigger more complex reset/boot controllers.

CHAPTER 4

# Bus Interface

## 4.1 Clocked Bus Model

All bus timing is described relative to the rising edge of CLKI. The CPU presents a bus request during the execution phase that needs external access. If no response is asserted, the CPU repeats the same request and does not advance to the next execution phase.

A no-wait-state bus access assumes that the selected external device can assert $\overline{\text{BACK}}$ and, for reads, drive valid data during the same CLKI interval in which $\overline{\text{BAS}}$ is asserted, so the CPU accepts the transfer on the next rising edge. A device that samples the request on one clock edge and cannot return $\overline{\text{BACK}}$ or read data until a later edge adds wait states; this is the normal way to attach slower synchronous memory or peripherals.

| Signal | Timing rule |
|---|---|
| A0–A23 | Valid whenever $\overline{\text{BAS}}$ is asserted. Stable until the request completes or aborts. |
| D0–D15 | Driven by the CPU only for writes. On reads, the selected device drives the bus and the CPU samples D0–D15 when $\overline{\text{BACK}}$ is accepted. |
| BRW | Valid whenever $\overline{\text{BAS}}$ is asserted. High requests a read; low requests a write. Stable until the request completes or aborts. |
| BAT0–BAT2 | Valid whenever $\overline{\text{BAS}}$ is asserted. Encodes the access type for the active request and is stable until completion or abort. |
| PROT | Reflects the status register P bit, except that it is deasserted for exception-vector fetches, and may be used with $\overline{\text{BAS}}$, BAT, and A0–A23 by external protection logic. |
| $\overline{\text{BAS}}$ | Asserted while a bus request is active. Remains asserted across wait states. |
| SYNC | Asserted only at phase 0 of an instruction fetch or of an exception-unit dispatch slot, and remains asserted if a wait state holds that phase. Deasserted in every other phase, in every other coprocessor slot (DMA and maths) and DMA continuation slot, and while the CPU is halted, waiting, or in reset. |

Table 4.1: CPU Bus Output Timing Rules

When $\overline{\text{BAS}}$ is deasserted there is no active bus cycle. External devices must ignore A0–A23, D0–D15, BRW, and BAT0–BAT2 unless $\overline{\text{BAS}}$ is asserted.

The CPU keeps driving A0–A23, BRW, and BAT0–BAT2 while $\overline{\text{BAS}}$ is deasserted, including while it is halted, waiting (**`WAIT`**), or in reset. BRW is high and BAT0–BAT2 is 000 (No I/O); this is the idle value. The value of A0–A23 while idle is architecturally undefined. The SIRC-1 has no bus-release mechanism: the CPU never three-states these signals, and $\overline{\text{HALT}}$ does not free the bus for another master.

## 4.2 Bus Cycle Completion

| Response | Effect When Sampled While $\overline{\text{BAS}}$ Is Asserted |
|---|---|
| $\overline{\text{BACK}}$ | Completes the request successfully. For reads, D0–D15 is sampled on the accepting clock edge. For writes, the write is externally committed. |
| $\overline{\text{BERR}}$ | Aborts the request and raises a bus fault. Read data is ignored and the CPU does not consider a write successfully committed. |
| $\overline{\text{BPER}}$ | Aborts the request and raises a bus protection fault. Read data is ignored and the CPU does not consider a write successfully committed. |
| No response | Inserts a wait state. The CPU remains in the current execution phase and keeps A0–A23, D0–D15 for writes, BRW, BAT0–BAT2, PROT, and $\overline{\text{BAS}}$ stable. |

Table 4.2: Bus Response Timing Rules

A bus cycle terminated by Bus Error or Bus Protection Error is architecturally unsuccessful. An external device must not commit any externally visible effect from a bus cycle for which it asserts the corresponding error signal. For a write, the device must not act on the data driven to it. For a read, the device must not cause any side effect that the access would otherwise have, for example a status register that clears on read. This is what makes Bus Fault and Bus Protection Fault safe to retry (see the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*): the CPU assumes that a bus cycle it aborted with one of these signals never happened.

$\overline{\text{BACK}}$, $\overline{\text{BERR}}$, and $\overline{\text{BPER}}$ are logically mutually exclusive. If more than one is asserted for the same request, $\overline{\text{BERR}}$ has priority, then $\overline{\text{BPER}}$, then $\overline{\text{BACK}}$.

## 4.3 Bus Cycle Types

| Cycle | CPU drives | External device drives | Completion |
|---|---|---|---|
| Read | A0–A23, BRW=1, BAT, $\overline{\text{BAS}}$, PROT | D0–D15 for the selected address | $\overline{\text{BACK}}$ samples data; $\overline{\text{BERR}}$/$\overline{\text{BPER}}$ aborts. |
| Write | A0–A23, D0–D15, BRW=0, BAT, $\overline{\text{BAS}}$, PROT | No data bus drive | $\overline{\text{BACK}}$ accepts write; $\overline{\text{BERR}}$/$\overline{\text{BPER}}$ aborts. |
| Wait state | Same values as the previous request cycle | Same ownership as the active request | Continues until $\overline{\text{BACK}}$, $\overline{\text{BERR}}$, or $\overline{\text{BPER}}$ is asserted. |
| No bus cycle | No active bus cycle | No device drives D0–D15 | CPU advances internally. |

Table 4.3: External Bus Cycle Ownership

## 4.4 DMA Bus Cycles

The optional DMA coprocessor performs bulk word transfers as ordinary single-word bus cycles. This section describes what an external device observes; the *SIRC-1 Architecture Reference Manual* describes the commands and their timing in the Coprocessor Instructions chapter. A CPU model without the DMA coprocessor never drives BAT 101 or 110.

- Phase 0 of the DMA slot is a setup cycle with no bus access: $\overline{\text{BAS}}$ is deasserted and the idle values of Section 4.1 apply.
- From phase 1, each DMA access is a separate bus cycle. $\overline{\text{BAS}}$ is asserted for the access, and the access is completed by its own $\overline{\text{BACK}}$, $\overline{\text{BERR}}$, or $\overline{\text{BPER}}$, exactly as for any other bus cycle.

- Addresses ascend by one word per access, for both positive and negative counts.
- BAT is 101 (DMA coprocessor read burst) for reads and 110 (DMA coprocessor write burst) for writes. BRW is high for reads and low for writes. A DMA read command uses only reads and a DMA write command uses only writes. A DMA copy command reads a chunk of up to seven words with BAT 101, then writes the chunk with BAT 110.
- PROT follows the status register P bit, as for any other access.
- SYNC is deasserted throughout the DMA slot and every DMA continuation slot.
- A wait state on a DMA access holds the phase and keeps the access signals stable, as for any other bus cycle. Each wait state adds exactly one cycle to the transfer.

Figure 4.1 shows a three-word DMA read with no wait states. The address labels `start`, `start+1`, and `start+2` are the first, second, and third word addresses of the transfer in ascending order.

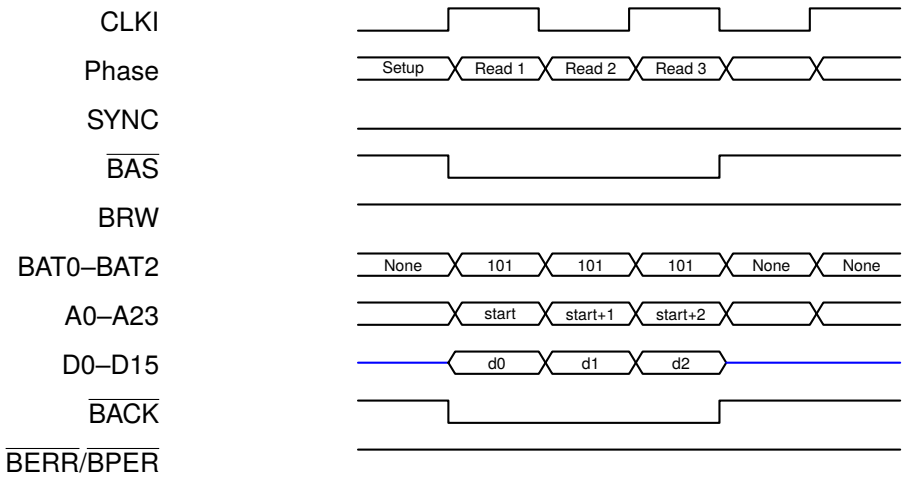



Figure 4.1: Three-Word DMA Read with No Wait States

## 4.5 Control Input Sampling

| Input | Sampling rule |
|---|---|
| $\overline{\text{BACK}}$, $\overline{\text{BERR}}$, $\overline{\text{BPER}}$ | Sampled while $\overline{\text{BAS}}$ is asserted. They complete or abort the active bus request. |
| $\overline{\text{IRQ1}}$–$\overline{\text{IRQ4}}$ | Level-sensitive and sampled every cycle, except during reset. The requesting device must hold the line asserted until its handler acknowledges it; a line is not latched while its own or a higher-priority handler runs. From the cycle in which reset is requested until the reset-vector fetch completes, no interrupt line, including $\overline{\text{NMI}}$, is sampled. |
| $\overline{\text{NMI}}$ | Edge-triggered. Fires once on assertion regardless of how long the pin is held; the pin must be released and reasserted to fire again. |
| $\overline{\text{HALT}}$ | Sampled at every phase 0. If asserted, the CPU stops at that phase 0 with the bus idle, which can fall between an instruction and its coprocessor dispatch slot, between DMA slots, or before a fault, interrupt, or reset dispatch. Halt latency is at most six cycles. The CPU resumes where it stopped when the input is deasserted; interrupt lines are still sampled while halted. |
| $\overline{\text{TRCE}}$ | Sampled at instruction boundaries. If asserted, the instruction being started is traced, except inside a fault handler. |
| $\overline{\text{RSTI}}$ | May be asserted at any time. It aborts the current instruction, exception dispatch, or bus wait and starts the reset sequence. |

Table 4.4: External Input Sampling Rules

CHAPTER 5

# Bus Timing Diagrams

## 5.1 Bus Cycle Diagrams

The timing diagrams below show cycle-level logical timing. Each labeled interval represents one CLKI cycle as observed between rising edges. Waveform rows show physical logic levels; for active-low signals, `L` means asserted and `H` means deasserted. Address, data, `BAT`, and phase rows are value bands rather than electrical setup/hold specifications. An empty band means that the row carries no defined value in that cycle.

Phase bands name the activity of the cycle. `IF high` and `IF low` are the high-word and low-word instruction-fetch cycles, and `Decode`, `Execute`, `Memory`, and `Writeback` are the pipeline phases of the same names, and `EA` is the effective-address calculation cycle. `EV high` and `EV low` are the high-word and low-word cycles of an exception-vector fetch, and `RV high` and `RV low` are those of the reset-vector fetch. `Mem` is the data-access cycle of a load or store, and `next` is the cycle that follows a bus transfer, in which the bus is idle. In a wait-state figure, `req` is the cycle in which the request is first presented, `wait` is a cycle in which no response is yet asserted, and `done` is the cycle in which the response completes the transfer. `Prev WB` is the writeback phase of the preceding instruction, and `Enter` is the first cycle of exception entry after the vector is fetched. `reset` is the cycle in which $\overline{\text{RSTI}}$ is first sampled asserted, and `hold` is a cycle of the reset-output hold. `BERR` and `BPER` label the abort cases.

`BAT` bands name the bus-access type. The diagrams use `IF` for instruction fetch, `DR` for data read, `DW` for data write, `EV` for exception vector fetch of vector $n$ at word addresses $2n$ and $2n+1$, and `None` for no bus access. `active` is the type of the access in progress when the abort is signalled. Address bands show `PC` and `PC+1` for the fetch addresses, `EA` for the effective address, and `addr` for the address of the access in progress. Data bands show the `high` or `low` word of a two-word transfer, or `data` for a single-word transfer.

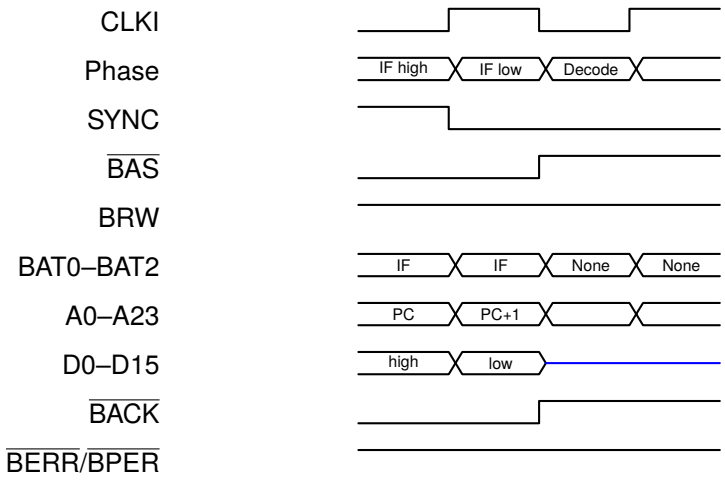



Figure 5.1: Instruction-Fetch High-Word and Low-Word Reads with No Wait States

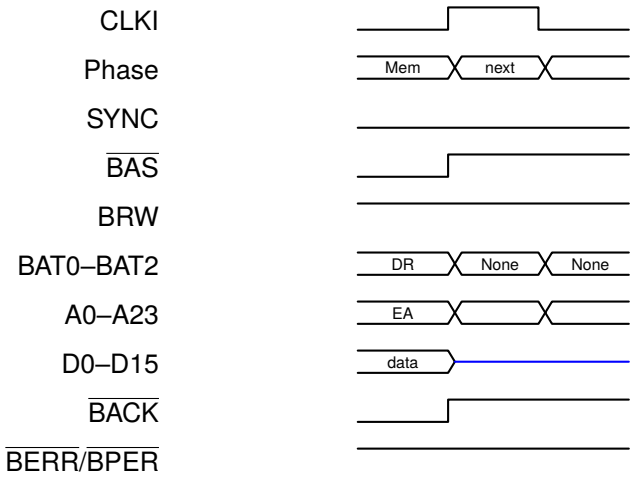



Figure 5.2: Data Read Cycle with No Wait State

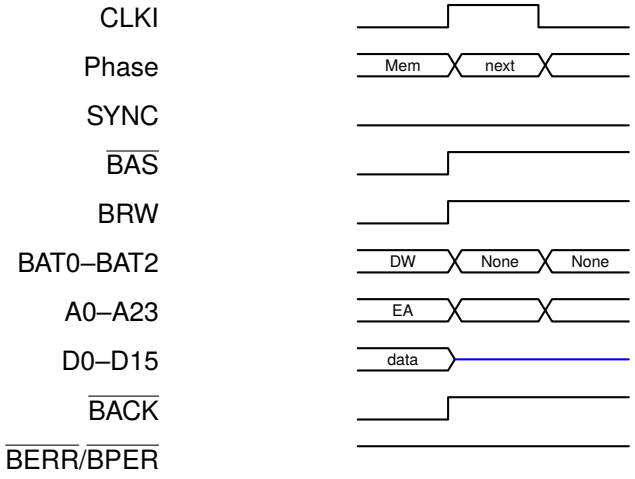



Figure 5.3: Data Write Cycle with No Wait State

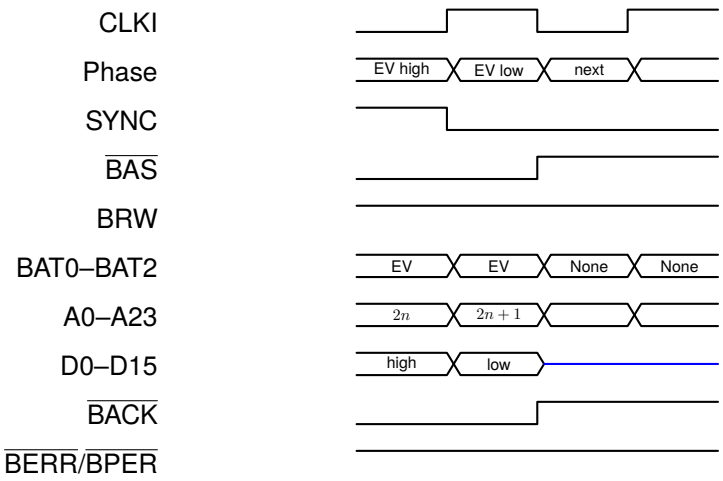



Figure 5.4: Exception-Vector Fetch with High-Word Then Low-Word Reads

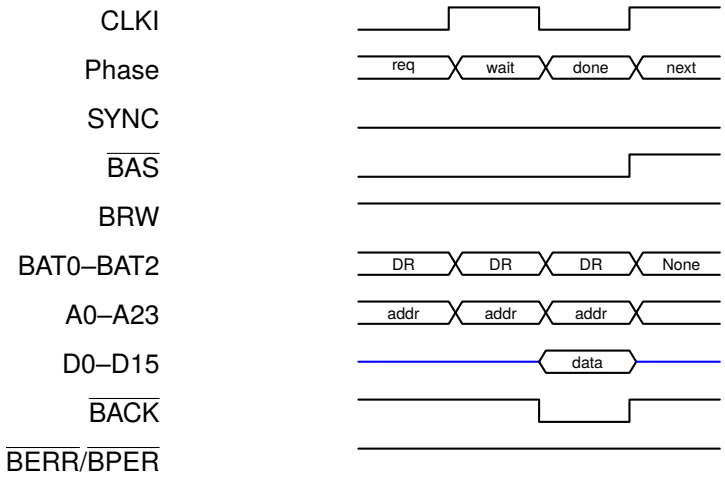



Figure 5.5: Wait-State Insertion when No Bus Response Is Asserted

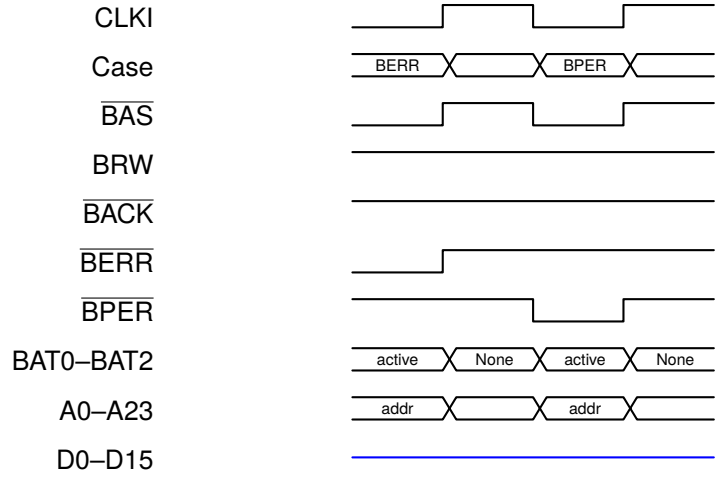



Figure 5.6: Bus-Error and Bus-Protection Abort Timing

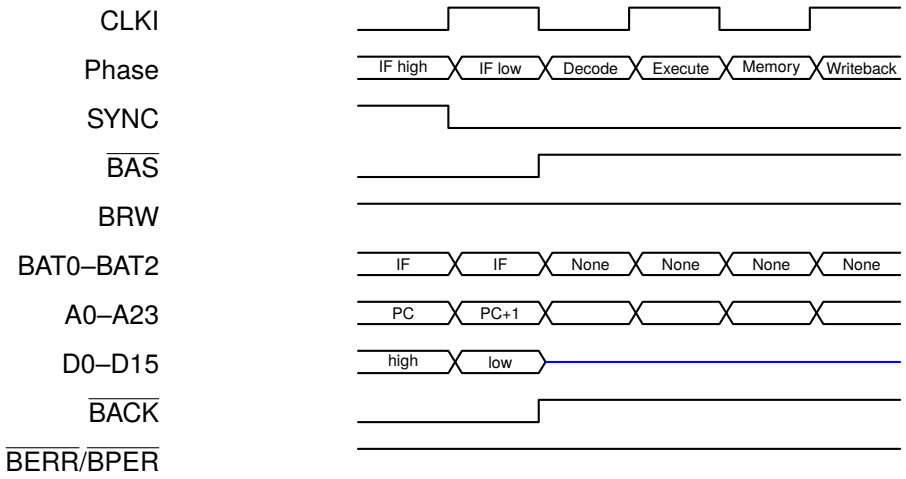



Figure 5.7: Complete Register-Only ALU Instruction with No Wait States

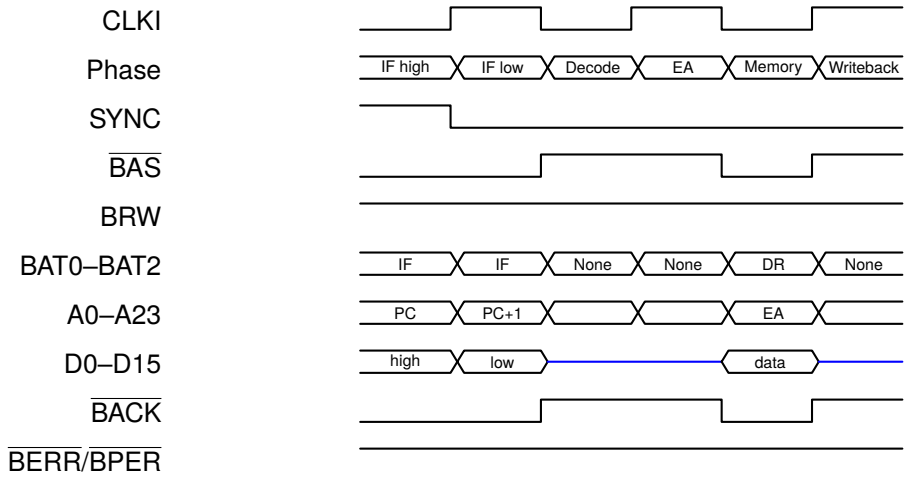



Figure 5.8: Complete LOAD Instruction with No Wait States

## 5.2 Reset and Interrupt Timing Examples

Figure 5.9 shows a hardware reset in which $\overline{\text{RSTI}}$ is asserted for one clock. The reset-output hold is six CLKI cycles in this case. The cycle in which $\overline{\text{RSTI}}$ is first sampled asserted counts as the first, and $\overline{\text{RSTO}}$ stays asserted for five further cycles after $\overline{\text{RSTI}}$ is sampled deasserted. An $\overline{\text{RSTI}}$ assertion of $n$ cycles therefore gives $n + 5$ cycles of $\overline{\text{RSTO}}$. A software reset and a reset-vector fetch retry each give a six-cycle hold. The reset-vector fetch does not begin until $\overline{\text{RSTI}}$ is deasserted and the reset-output hold has completed.

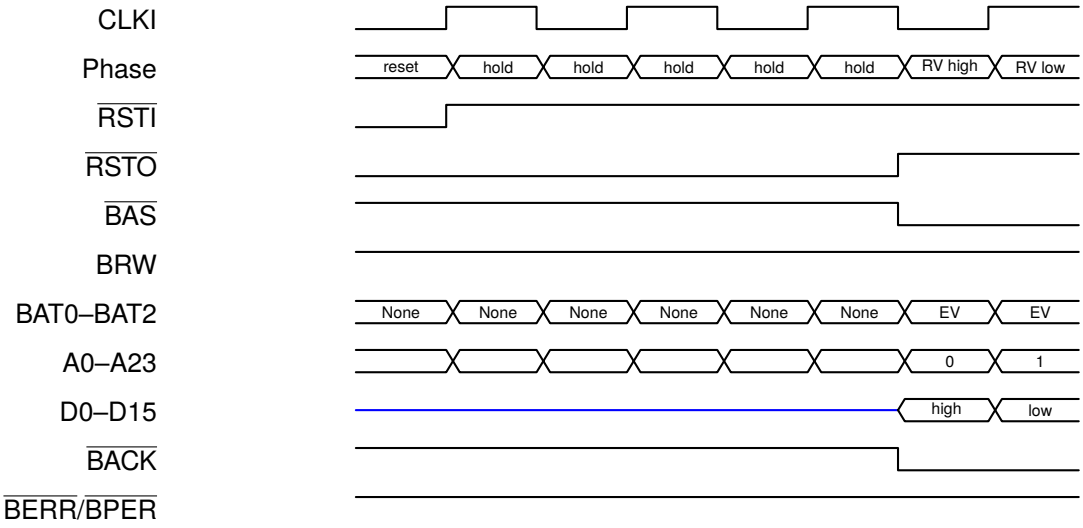



Figure 5.9: Hardware Reset with One-Cycle $\overline{\text{RSTI}}$ Assertion and Reset-Vector Fetch

Figure 5.10 shows an enabled maskable interrupt asserted before an instruction boundary. The CPU samples the interrupt lines on every clock cycle, so the request is latched in the cycle it is first seen, but it is dispatched only at the next phase 0 at which no coprocessor call or command is pending or active. A request first sampled at the rising edge that starts phase 0 is dispatched in that phase 0. The following exception-unit dispatch fetches the selected hardware-interrupt vector instead of fetching the next processing-unit instruction. A request latched partway through an instruction waits for the next such phase 0. The figure holds $\overline{\text{IRQ1}}$–$\overline{\text{IRQ4}}$ asserted throughout because the requesting device must hold the line asserted until its handler acknowledges it.

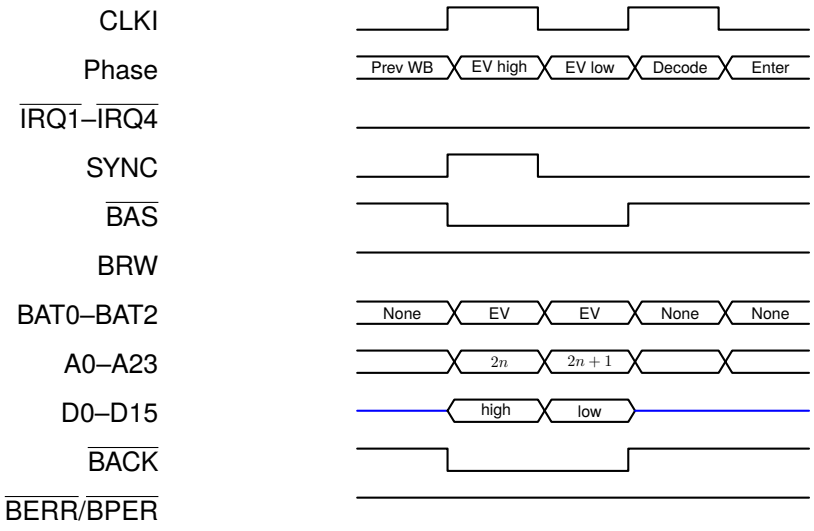



Figure 5.10: Enabled IRQ Latched Before an Instruction Boundary and Dispatched with No Wait States

CHAPTER 6

# Electrical Characteristics

The only electrical requirement currently specified is the supply: VCC1 and VCC2 must be connected to 5 V $\pm$ 5% and GND1 and GND2 to the 0 V rail (see Chapter 3).

The bus timing in this manual is cycle-level logical timing, measured in CLKI cycles. The following characteristics are not yet specified and will be added to this chapter when they are:

- Input and output voltage levels
- Clock frequency range
- Setup, hold and propagation delay times
- Fanout and maximum board loading
- Supply current and power dissipation
- Package dimensions and thermal characteristics

Until then, designs must not assume any value for these characteristics.

# SIRC-1 Hardware Reference Manual

This manual describes the SIRC-1 as a physical device: the package and pinout, what every signal does, and the bus protocol the CPU speaks to memory and peripherals. It is written for the engineers who design boards, memory controllers and peripherals around the chip. What the processor does with the data once it has it is the subject of the companion SIRC-1 Architecture Reference Manual.

**This manual contains:**

- The 64-pin DIP package and pin-by-pin signal descriptions
- The clocked bus protocol: transfers, wait states, bus errors and bus protection
- Cycle-level timing diagrams for instruction fetch, data access, reset and interrupts
- The rules for sampling the interrupt, halt, trace and reset inputs

SILICON INTEGRATED RESEARCH CORPORATION
1400 Fabrication Way, Milpitas Flat, CA
Printed in the United States of America

First Printing, 1989 • Version 1.0